Foundations of Artificial neural network

[Artificial neural network]

$$a^{(l)} = \sigma!\left(W^{(l)}a^{(l-1)} + b^{(l)}\right)$$

1.0 Content Block

Concept drift The statistical properties of input data may change over time, a phenomenon known as concept drift or non-stationarity. Drift can reduce predictive accuracy and lead to unreliable or biased decisions if it is not detected and corrected. In practice, this means that the model's accuracy in deployment may differ substantially from levels observed during training. Several strategies have been developed to monitor neural networks for drift and degradation:

2.0 Content Block

Network design The choice of model depends on the data and the application. Models that work well with textual data are typically not the best choice for image data, etc. An important element is which training/learning the model uses. Neural architecture search (NAS) uses machine learning to automate NN design. NAS has yielded networks that compare well with hand-designed systems. The basic algorithm is to propose a candidate model, evaluate it against a dataset, and use the results as feedback to teach the NAS network. Efforts include AutoML and AutoKeras. scikit-learn library provides functions to help with building a deep network from scratch. Hyperparameters are design choices (they are not learned).

3.0 Content Block

Historical foundations and the Dartmouth proposal Artificial neural networks were identified as a promising direction for artificial intelligence research in the 1955 proposal for the Dartmouth Summer Research Project on Artificial Intelligence. Neural network models initially faced major limitations. Hardware constraints limited network size and training efficiency, while theoretical understanding of learning algorithms remained incomplete. Many models used single-layer perceptrons, which were restricted to solving linearly separable problems. These limitations were highlighted in the book Perceptrons by Marvin Minsky and Seymour Papert, which deflated interest during the late 1960s and 1970s.

4.0 Content Block

Function approximation, or regression analysis, (including time series prediction, fitness approximation, and modeling) Data processing (including filtering, clustering, blind source separation, and compression) Nonlinear system identification and control (including vehicle control, trajectory prediction, adaptive control, process control, and natural resource management) Pattern recognition (including radar systems, face identification, signal classification, novelty detection, 3D reconstruction, object recognition, and sequential decision making) Sequence recognition (including gesture, speech, and handwritten and printed text recognition) Sensor data analysis (including image analysis) Robotics (including directing manipulators and prostheses) Data mining (including knowledge discovery in databases) Finance (such as ex-ante models for specific financial long-run forecasts and artificial financial markets) Quantum chemistry General game playing Generative AI Data visualization Machine translation Social network filtering E-mail spam filtering Medical diagnosis Disaster response NNs have been used to diagnose cancers and to distinguish highly invasive cancer cell lines from less invasive lines using only cell shape data. NNs have been used to accelerate reliability analysis of infrastructures subject to natural disasters and to predict settling in building foundations. They are used to mitigate flooding by modelling rainfall-runoff. NNs have

been used for building black-box models in geoscience: hydrology, ocean modelling and coastal engineering, and geomorphology. NNs have been employed in cybersecurity, with the objective to discriminate between legitimate and malicious activities. For example, machine learning has been used for classifying Android malware, for identifying domains belonging to threat actors and for detecting URLs posing a security risk. Research is underway on penetration testing, for detecting botnets, credit cards frauds, and network intrusions. NNs have been proposed as a tool to solve partial differential equations in physics and simulate the properties of many-body open quantum systems. In brain research NNs have studied short-term behavior of individual neurons, the dynamics of neural circuitry arise from interactions between individual neurons and how behavior can arise from abstract neural modules that represent complete subsystems. Studies considered long-and short-term plasticity of neural systems and their relation to learning and memory from the individual neuron to the system level. NNs show promise in profiling a user's interests from photos and discovering new stable materials by efficiently predicting the total energy of crystals.

5.0 Content Block

Computational power The multilayer perceptron is a universal function approximator, as proven by the universal approximation theorem. However, the proof does not specify the number of neurons required, the network topology, the weights or the learning parameters. A recurrent architecture with rational-valued weights (as opposed to full precision real number-valued weights) has the power of a universal Turing machine, using a finite number of neurons and linear connections. Further, the use of irrational values for weights results in a machine with super-Turing power.

6.0 Content Block

Learning can be either stochastic or batch. Stochastic learning creates a weight adjustment for each sample. In batch learning, weights are adjusted based on a batch of inputs, accumulating errors over the batch. Stochastic learning introduces "noise" into the process, using the local gradient calculated from one data point; this reduces the chance of the network getting stuck in local minima. However, batch learning typically yields a faster, more stable descent to a local minimum, since each update is performed in the direction of the batch's average error. A common compromise is to use "mini-batches", small batches with samples in each batch selected stochastically from the entire data set.

7.0 Content Block

Training NNs require millions of training samples to become functional. As of 2026, training a commercial LLM (GPT, Grok, Gemini) typically required hundreds of thousands of computers, and cost 10s of millions of dollars.

8.0 Content Block

Recurrent neural networks One source of RNN was statistical mechanics. In 1972, Shun'ichi Amari proposed to modify the weights of an Ising model by Hebbian learning rule as a model of associative memory, adding in learning. This was popularized as the Hopfield network by John Hopfield (1982). Another origin of RNN was neuroscience. The word "recurrent" is used to describe loop-like structures in anatomy. In 1901, Cajal observed "recurrent semicircles" in the cerebellar cortex. Hebb considered "reverberating circuit" as an explanation for short-term memory. The McCulloch and Pitts paper (1943) considered neural networks that contain cycles, and noted that the current activity of such networks can be affected by activity indefinitely far in the past. In 1982 Crossbar Adaptive Array, a recurrent neural network with an array architecture (rather than a multilayer perceptron architecture), used direct recurrent connections from the output to the supervisor (teaching) inputs. In addition to computing actions (decisions), it computed internal state evaluations (emotions) of consequence situations. Eliminating the external supervisor, it introduced the self-learning method in neural networks. In cognitive psychology, the American Psychologist journal in the early 1980s debated the relation between cognition and emotion.

Social psychologist Robert Zajonc in 1980 stated that emotion is computed first and is independent from cognition, while Richard Lazarus in 1982 stated that cognition is computed first and is inseparable from emotion. It was an example of a debate where an RNN contributed to an issue and also addressed cognitive psychology. The Jordan network (1986) and the Elman network (1990) applied RNN to study cognitive psychology. In the 1980s, backpropagation did not work well for deep RNNs. In 1991, Jürgen Schmidhuber proposed the "neural sequence chunker" or "neural history compressor" which introduced self-supervised pre-training (the "P" in ChatGPT) and neural knowledge distillation. In 1993, a neural history compressor system solved a "Very Deep Learning" task that required more than 1000 layers in an RNN unfolded in time. In 1991, Sepp Hochreiter's diploma thesis identified and analyzed the vanishing gradient problem and proposed recurrent residual connections to solve it. He and Schmidhuber introduced long short-term memory (LSTM), which set accuracy records in multiple application domains. This was not the ultimate version of LSTM, which required the forget gate, and was introduced in 1999. It became the default choice for RNN architecture. During 1985–1995, inspired by statistical mechanics, several architectures and methods were developed by Terry Sejnowski, Peter Dayan, Geoffrey Hinton, and others, including the Boltzmann machine, restricted Boltzmann machine, Helmholtz machine, and the wake-sleep algorithm. These were designed for unsupervised learning of deep generative models.